

Software Design Document (SDD) This section should be elaborated further with architecture diagrams, sequence flows, validation rules, error handling, implementation details, and examples in a full production SDD.

AI Test Case Generator & QA Assistant Version 1.0 This section should be elaborated further with architecture diagrams, sequence flows, validation rules, error handling, implementation details, and examples in a full production SDD.

1. Introduction

Purpose: Design and implement an AI-powered web application that generates software test cases from uploaded requirement documents and provides a project-specific QA assistant using Retrieval-Augmented Generation (RAG). Scope: Users create projects, upload PDF/DOCX requirement documents, generate AI-powered test cases, ask questions about project documents, and export results. The system is intended for QA engineers, software testers, students, and development teams. This section should be elaborated further with architecture diagrams, sequence flows, validation rules, error handling, implementation details, and examples in a full production SDD.

2. Objectives

Reduce manual effort in writing test cases, improve requirement understanding, centralize project artifacts, provide contextual AI answers, and support reusable project knowledge. This section should be elaborated further with architecture diagrams, sequence flows, validation rules, error handling, implementation details, and examples in a full production SDD.

3. Technology Stack

Backend: ASP.NET Core 10 Web API Frontend: Angular 21 Database: MySQL 8 + Entity Framework Core 10 Authentication: JWT + Refresh Tokens AI: Claude API Embeddings: OpenAI Embedding API Vector Store: ChromaDB Libraries: PdfPig, Open XML SDK, QuestPDF, ClosedXML, AutoMapper, FluentValidation, Serilog. This section should be elaborated further with architecture diagrams, sequence flows, validation rules, error handling, implementation details, and examples in a full production SDD.

4. High Level Architecture

Angular communicates with ASP.NET Core REST APIs. Business logic is implemented in Services. Entity Framework Core accesses MySQL. Uploaded documents are processed into chunks, converted to embeddings, stored in ChromaDB, and later retrieved during AI chat or test case generation. Claude produces natural language responses. This section should be elaborated further with architecture diagrams, sequence flows, validation rules, error handling, implementation details, and examples in a full production SDD.

5. Modules

Authentication: Register, Login, Refresh Token, Logout. Project Management: Create, update, delete and manage multiple projects. Document Management: Upload, download, delete, extract text, chunk documents, generate embeddings. AI Chat: RAG-powered project assistant with chat history. Test Case Generator: Generates positive, negative, edge and regression test cases. Export: Export generated test cases to PDF and Excel. Dashboard: Project statistics and summaries. User Management: Profile management. This section should be elaborated further with architecture diagrams, sequence flows, validation rules, error handling, implementation details, and examples in a full production SDD.

6. Database Design

Tables: Users Projects Documents DocumentChunks TestCases ChatHistory Relationships:
User→Projects (1:M) Project→Documents (1:M) Document→DocumentChunks (1:M)
Project→TestCases (1:M) Project→ChatHistory (1:M) User→ChatHistory (1:M). This section should be elaborated further with architecture diagrams, sequence flows, validation rules, error handling, implementation details, and examples in a full production SDD.

7. Backend Design

Folder Structure: Controllers Services Interfaces DTOs Entities Data Mappings Middleware Validators Extensions Controllers handle HTTP requests, Services contain business logic, DTOs isolate API contracts, Entities map database tables, Interfaces enable dependency injection. This section should be elaborated further with architecture diagrams, sequence flows, validation rules, error handling, implementation details, and examples in a full production SDD.

8. AI & RAG Design

Workflow: Upload PDF/DOCX → Extract Text → Chunk Text → Generate Embeddings → Store in ChromaDB. When a user asks a question, an embedding is created, similar chunks are retrieved from ChromaDB, a prompt is constructed, Claude generates an answer, and chat history is stored. This section should be elaborated further with architecture diagrams, sequence flows, validation rules, error handling, implementation details, and examples in a full production SDD.

9. Test Case Generation

The system retrieves relevant document chunks using semantic search and instructs Claude to generate structured positive, negative, edge and regression test cases including title, priority, preconditions, steps and expected results. Generated cases are parsed and stored in MySQL. This section should be elaborated further with architecture diagrams, sequence flows, validation rules, error handling, implementation details, and examples in a full production SDD.

10. APIs

AuthController, ProjectController, DocumentController, AIChatController, TestCaseController, ExportController, DashboardController and UserController expose REST endpoints secured using JWT. This section should be elaborated further with architecture diagrams, sequence flows, validation rules, error handling, implementation details, and examples in a full production SDD.

11. Security

JWT authentication, BCrypt password hashing, refresh tokens, authorization, FluentValidation, CORS, Serilog logging and global exception middleware provide application security. This section should be elaborated further with architecture diagrams, sequence flows, validation rules, error handling, implementation details, and examples in a full production SDD.

12. Deployment

Frontend and backend are maintained in separate Git repositories. MySQL stores relational data while ChromaDB stores vectors. Configuration is maintained through appsettings.json and environment variables. This section should be elaborated further with architecture diagrams, sequence flows, validation rules, error handling, implementation details, and examples in a full production SDD.

13. Future Enhancements

Role-based access control, Jira integration, Azure DevOps integration, Playwright/Selenium script generation, requirement traceability matrix, AI bug report generation, collaboration features and model abstraction for multiple LLM providers. This section should be elaborated further with

architecture diagrams, sequence flows, validation rules, error handling, implementation details, and examples in a full production SDD.

14. Conclusion

The proposed architecture is modular, scalable and maintainable. It separates AI services, vector search, business logic and persistence while enabling future expansion with minimal changes. This section should be elaborated further with architecture diagrams, sequence flows, validation rules, error handling, implementation details, and examples in a full production SDD.